

Infraestructura como código, Ansible y NixOS

Hecho por:

Sergio García-Morales Coca

Alejandro Guillén Plazuelo

Iván Labrador Eugenio

Profesor:

Alejandro Marcos De Rama

ÍNDICE:

1. Introducción.....	3
2. ¿Qué es la infraestructura como código?.....	3
3. Modelos de configuración de un sistema.....	3
4. Ansible ¿Qué es?.....	5
5. Cómo funciona Ansible (o Ansible playbooks, decidir).....	5
6. Ventajas de Ansible.....	6
7. Nix, Nix y Nix.....	6
8. NixOS, qué es.....	6
9. ¿Cómo funciona NixOS?.....	7
10. Nix Store, shells temporales y entornos de desarrollos.....	8
11. ¿Por qué NixOS?.....	8
12. Diferencias clave entre Ansible y NixOS.....	9
13. Conclusiones.....	9
Bibliografía.....	10

1. Introducción

A la hora de configurar sistemas en masa nos encontramos rápidamente con un gran problema, la eficiencia. Para solventar este problema utilizamos herramientas de orquestación o que nos permiten crear sistemas reproducibles y desplegarlos de forma simultánea. Algo que comparten estas herramientas es poder definir la infraestructura de desde un equipo hasta de una flota completa con ordenadores, servidores y otros dispositivos a través de código.

2. ¿Qué es la infraestructura como código?

La infraestructura como código es la práctica de tratar la infraestructura como software, lo que nos permite “programar” las máquinas en lugar de configurarlas una a una manualmente, lo que elimina errores ya que deja de ser un proceso manual para cada una de las máquina que queramos configurar y vuelve este proceso mucho más eficiente. Además reduce un montón de costes para una empresa, acelera los flujos de trabajo ya que, en lugar de tener que montar manualmente un equipo cada vez que se necesite, se despliega con este código que actúa como el propio sistema y todo queda automatizado y libre de errores si se ha depurado el código anteriormente. Es casi infinitamente escalable y tenemos consistencia absoluta, siempre y cuando utilicemos un modelo de configuración adecuado.

3. Modelos de configuración de un sistema

A la hora de configurar sistemas a escala es importante

- *Divergente*: Es la forma tradicional de configurar un sistema. Consiste en configurar el sistema a mano, lo cual implica inconsistencia y una muy alta probabilidad de cometer errores. Además hace que la reproducibilidad sea casi nula y a medida que se modifica el sistema, se aleja del estado deseado en el que se había configurado.

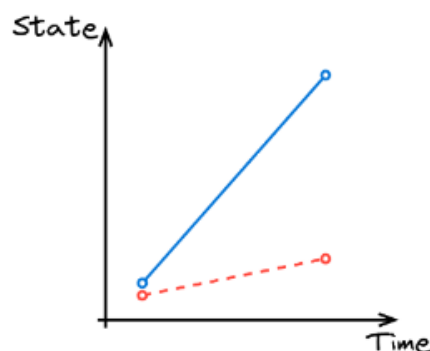


Figure 15: Divergent

- *Convergente*: Este modelo intenta traer sistemas a un estado común y deseado, lo que implica consistencia pero no necesariamente garantiza que el estado de las máquinas sea el mismo. El modelo se basa en comparar el estado actual de un sistema con el estado deseado y realizar acciones para rectificar la configuración a la deseada pero siempre van a existir remanentes de las configuraciones anteriores aunque sean menores, lo que implica variabilidad entre los sistemas y hace que no acaben siendo iguales, sin embargo es una gran mejora ante la divergencia del modelo anterior y es muchas veces una solución suficiente. Ansible y Terraform, dos de herramientas muy estandarizadas en la industria, siguen este modelo.

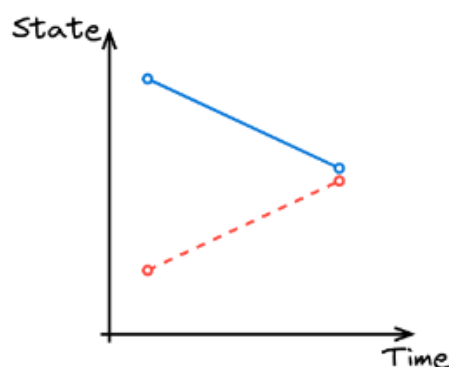


Figure 16: Convergent

- *Congruente*: Es el modelo que tiene la configuración más estricta y a la vez más reproducible. Los sistemas no siguen una serie de instrucciones para alcanzar un estado deseado, sino que se construyen desde cero siguiendo el código que lo configura. Esto hace que el sistema resultante sea idéntico a la configuración deseada y a la vez inmutable, es decir, que solo se puede modificar si el propio código que configura al sistema se modifica y se vuelve a construir, quedando retratado en el código cada cambio y eliminando cualquier tipo de divergencia con la configuración deseada o variabilidad entre los sistemas. Proporciona una consistencia, reproducibilidad y confianza como ninguna otra pero son menos flexibles a entornos con cambios rápidos por la necesidad de volver a construirse por cada set de cambios que hagamos en su configuración, aunque las herramientas que implementan este modelo utilizan técnicas para minimizar este inconveniente. Nix, NixOS y Guix siguen este modelo.

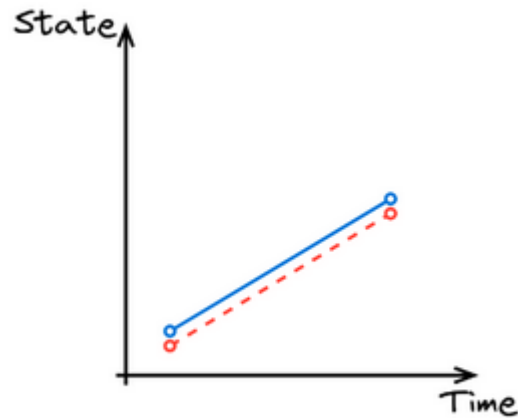


Figure 17: Congruent

4. Ansible ¿Qué es?

Ansible es un software de orquestación open source desarrollado por Michael DeHann en 2012, aunque en 2015 fue adquirido por Red Hat.

Ansible permite automatizar cientos de tareas necesarias para la puesta en marcha y mantenimiento de una red de equipos siguiendo un modelo convergente: preparación de infraestructura, implementación de aplicaciones u organización de los sistemas son ejemplos de la gran cantidad de opciones que este software nos ofrece. Una manera de utilizar Ansible para realizar estos despliegues es con los Ansible Playbooks, que actúan como un “manual de instrucciones” con una serie de pasos a seguir para alcanzar la configuración deseada.

Es importante diferenciar la versión comunitaria de Ansible de su versión comercial Red Hat Ansible Automation Platform. Esta segunda versión cuenta con soporte para empresas y ciertas funciones diseñadas para ayudar a las mismas, aunque en este documento no se entra en detalles de la versión comercial.

5. Cómo funciona Ansible (o Ansible playbooks, decidir)

Ansible trabaja de forma totalmente independiente, es decir, no es necesario instalar absolutamente nada en los equipos o nodos que queremos gestionar. Esto es posible gracias a la utilización del protocolo SSH para conectarse y ejecutar las tareas en los nodos gestionados.

Ansible se encarga de instalar módulos, pequeños programas que contienen las tareas automatizadas, en los equipos gestionados, y aunque existen una gran variedad de módulos predeterminados, Ansible nos ofrece la posibilidad de crear nuestros módulos personalizados, permitiendo una gran versatilidad a los administradores de las redes que utilicen este software.

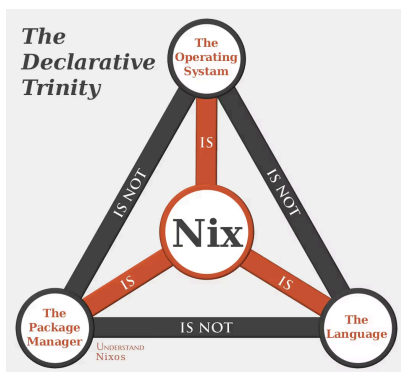
Estos módulos personalizados pueden ser escritos en cualquier lenguaje de programación que se convierta en JSON, como por ejemplo Python o Bash. En caso de querer automatizar tareas en Windows, también pueden escribirse módulos en PowerShell.

6. Ventajas de Ansible

- Agentless: no requiere instalarse software agente, se conecta mediante SSH.
- Estúpidamente fácil de aprender.
- Opensource
- Tiene una forma de testear los cambios sin ejecutarlos
- Python-based
- Gratis
- Push based model: Ansible empuja los scripts a los servidores en vez de pullear los cambios desde el servidor.
- Hot line command y playbook: puedes ejecutar comandos de linux directamente o hacer un script con instrucciones.

7. Nix, Nix y Nix

Cuando hablamos de Nix ¿De qué hablamos?



Cuando nos referimos a Nix podemos hablar de tres elementos diferenciados pero que trabajan de forma interconectada en su mayoría:

- Nix, el lenguaje de programación sobre el que se construye el resto de bloques
- Nix, el administrador de paquetes
- Nix, el sistema operativo, mejor conocido como NixOS y que implementa los anteriores elementos bajo su propia distribución de Linux.

En el documento se hablará sobre NixOS, cómo funciona y su utilidad.

8. NixOS, qué es

NixOS es una distribución de linux independiente, es decir, que no está basada en ninguna otra, y que adopta la filosofía de Infraestructura como Código al completo, creando como resultado un sistema operativo configurado de forma declarativa, inmutable y completamente reproducible. ¿Y qué hace NixOS para que sea así de reproducible, cómo funciona por dentro?

9. ¿Cómo funciona NixOS?

Primero de todo hay que entender cómo se configura. NixOS se configura mediante ficheros de configuración escritos en su propio lenguaje de programación, Nix. En estos ficheros se definen las características, servicios, aplicaciones, etc., que se quiere que tenga un sistema. Un ejemplo de configuración puede ser tan simple como el siguiente extracto de código:

```
{
  boot.loader.grub.enable = true;
  networking.hostName = "ordenador-nixos"
  services.sshd.enable = true;
  environment.systemPackages = [
    pkgs.ansible
  ];
}
```

En este extracto de código se configura el bootloader del sistema para utilizar grub, definimos el hostname del sistema, se activa el servicio de ssh y se instala el paquete ansible en el sistema. Sin embargo, esta configuración no va a tener efecto alguno hasta que el sistema no se reconstruya con esta configuración.

nixos-rebuild switch

Al ejecutar el comando se reconstruye el sistema utilizando la configuración que hemos creado anteriormente, pero a contrario que Ansible, que sigue un modelo convergente que intenta corregir el estado de la máquina al estado deseado, este proceso de reconstrucción sigue un modelo congruente, lo que significa que descarta toda la configuración anterior, creando un sistema “vacío”, para luego configurarlo de cero utilizando la nueva configuración, se crea una nueva “generación. Esto asegura que cada elemento de nuestro sistema haya sido definido explícitamente sin ningún tipo de variabilidad y que dada la misma configuración de entrada el resultado siempre sea el mismo en todas las máquinas donde se construya esta configuración. Además, cada vez que se reconstruye el sistema se guarda la generación anterior y gracias a la forma de NixOS de gestionarlas significa que si en algún punto algo sale mal, se pueda hacer un rollback a una configuración o generación anterior en vivo y sin tener que reiniciar el equipo.

10. Nix Store, shells temporales y entornos de desarrollos

NixOS no cumple con el estándar FHS, hace las cosas a su manera y este es su mayor fuerte. Todos los paquetes de NixOS están aislados en su propio directorio dentro de la Nix Store que se encuentra en `/nix/store`. Por ejemplo, la ruta al paquete de firefox sería:

```
/nix/store/5rnfzla9kcx4mj5zdc7nlnv8na1najvg-firefox-3.5.4/
```

Donde “5rnfz...” es un hash que se genera con el fichero de configuración de entrada al paquete, lo que significa que los paquetes nunca son sobrescritos. Esto introduce un gran problema ya que se estarían almacenando decenas de versiones de un mismo paquete, cada una ocupando espacio pero NixOS lo mitiga realizando enlaces a archivos duplicados entre paquetes y ejecutando un recolector de basura para eliminar paquetes antiguos y que ya no son necesarios. A su vez, estos paquetes están aislados o “sandboxeados”, siendo cada paquete un pequeño contenedor, lo que significa que si funciona en un sistema, funcionará en cualquier otro.

Todo en NixOS es un paquete escrito en Nix, incluso nuestra configuración que, al ser reconstruida, se guarda en la Nix Store. Esto mismo es lo que hace que volver a una generación anterior de nuestro sistema de forma instantánea sea una posibilidad ya que NixOS simplemente “activa” la generación anterior desde la Nix Store junto con los paquetes y sus versiones instaladas en esa misma generación.

Nix-shell es una herramienta muy potente de Nix, el gestor de paquetes, que nos permite crear entornos temporales una shell, pudiendo descargar programas, configurar variables de entorno, hacer configuraciones, etc., que al cerrar la shell serán descartados y funciona, de nuevo, gracias a la Nix Store.

Esto tiene dos grandes utilidades:

- Probar programas sin ser “instalados”. El programa es descargado al sistema pero no activado, lo que significa que se ejecuta únicamente mientras la shell esté abierta y al cerrarla el programa desaparecerá del sistema
- Crear entornos de desarrollo. Junto con otra herramienta llamada *direnv* podemos crear una shell que de active al entrar a un directorio en específico que contenga dependencias, configuraciones y programas para desarrollar en un lenguaje de programación específico sin necesidad de instalarlo en el sistema, creando entornos aislados y eliminando completamente el conflicto de dependencias.

11. ¿Por qué NixOS?

- Reproducibilidad asegurada
- Actualizaciones atómicas, se hacen o no se hacen, no hay punto medio
- Elimina el miedo a cambios, se puede hacer rollback en cualquier momento
- Flujo de trabajo con control de versiones
- Elimina los conflictos de dependencias
- Todo es Nix, es un ecosistema unificado y consistente

12. Diferencias clave entre Ansible y NixOS

Ambas opciones permiten crear infraestructuras robustas utilizando el poder de la infraestructura como código, pero ambas lo hacen con un enfoque diferente y único, llevando a cubrir cada una un gran rango de casos de uso en los que algunos de ellos se superponen y otros no.

Ansible es una herramienta enfocada en la orquestación de flotas de equipos y sistemas, sea 1, 10 o 100 de ellos. Es completamente agnóstico al sistema operativo que se quiera configurar ya que funciona de forma remota, lanzando comandos a los sistemas objetivos, es fácil de aprender y se integra con una gran cantidad de lenguajes de programación, todos los que se puedan traducir a json y está mucho más estandarizado. Su objetivo es llevar una serie de máquinas a un estado común, utilizando un modelo convergente para configurar los sistemas y por ende puede haber ciertas variaciones entre ellos sin embargo, para los entornos en los que se utiliza, estas variaciones no suelen introducir grandes problemas. Ansible acaba funcionando como un libro de instrucciones para que un sistema alcance un estado deseado.

Por el contrario, NixOS funciona de una forma mucho más purista y obliga a utilizar un sistema basado en Linux, además de tener que adoptar todo el ecosistema de Nix, aunque cabe aclarar que el Nix el gestor de paquetes también se encuentra disponible para WSL, MacOS, diversas distribuciones de Linux y Docker. Tiene herramientas muy poderosas y se centra en la reproducibilidad pura de sistemas, pudiendo montar miles de equipos exactamente iguales al 100% con un solo fichero de configuración y sin introducir ningún tipo de variabilidad entre ellos. Su lenguaje, Nix es difícil de aprender y dominar y su uso es muy específico. Sigue un modelo congruente, asegurándose de que el sistema sea inmutable y su estado sea siempre el mismo dada la misma configuración como entrada. NixOS y sus configuraciones no funcionan como instrucciones, si no como un manifiesto del estado final de la máquina.

13. Conclusiones

La infraestructura como código es una forma de gestionar una infraestructura de sistemas y equipos muy potente y realmente útil que puede ser implementada de muchas maneras diferentes. Además de Ansible y NixOS existen muchísimas herramientas que también abarcan los mismos problemas e incluso se especializan en ciertos nichos como Terraform, Chef, Puppet o Guix.

Para poder decidir qué herramienta utilizar no es tan fácil como comparar su ventajas y desventajas, ya que en la mayoría de los casos hacer un estudio de la infraestructura y realmente comprobar qué se adapta mejor a las necesidades de la misma va a ser mejor opción que comparar una lista de positivos y negativos y ver cual sale con mejor balance. Cada infraestructura tiene sus propias necesidades y objetivos que cumplir y cada herramienta va a ayudar a la infraestructura a alcanzarlos de una forma diferente, sea un instituto, una oficina o una flota de servidores que van a dar un servicio.

Bibliografía

Holdsworth, J. (2025). What is infrastructure as code (IaC)?. IBM.

<https://www.ibm.com/think/topics/infrastructure-as-code>

Dellaiera, P. (2026). Reproducibility in Software Engineering. Zenodo.

<https://doi.org/10.5281/zenodo.18701980>

Dellaiera, P. (2025). Refactoring My Infrastructure As Code Configurations. not-a-number.

<https://not-a-number.io/2025/refactoring-my-infrastructure-as-code-configurations/>

Ansible Community. (2026). Ansible Community Documentation.

<https://docs.ansible.com/>

Finger, J (2025). Nix - Death by a thousand cuts. Jono's Corner.

<https://www.dgt.is/blog/2025-01-10-nix-death-by-a-thousand-cuts/>

NixOS Community (2026). How Nix Works. NixOS.org

<https://nixos.org/guides/how-nix-works/>

Zemb, P (2025). Three Years of Nix and NixOS: The Good, the Bad, and the Ugly. Pierre Zemb's Blog.

<https://pierrezemb.fr/posts/nixos-good-bad-ugly>

Nix Community (2025). Nix reference manual. Nix Manual.

<https://nix.dev/manual/nix/2.28/>